



如果函数有可能用到某全局变量，则不宜再定义一个同名的局部变量。

2.2.4 节练习

练习 2.13: 下面程序中 `j` 的值是多少?

```
int i = 42;
int main()
{
    int i = 100;
    int j = i;
}
```

练习 2.14: 下面的程序合法吗? 如果合法, 它将输出什么?

```
int i = 100, sum = 0;
for (int i = 0; i != 10; ++i)
    sum += i;
std::cout << i " " << sum << std::endl;
```

2.3 复合类型

复合类型 (compound type) 是指基于其他类型定义的类型。C++ 语言有几种复合类型, 本章将介绍其中的两种: 引用和指针。

与我们已经掌握的变量声明相比, 定义复合类型的变量要复杂很多。2.2 节 (第 38 页) 提到, 一条简单的声明语句由一个数据类型和紧随其后的一个变量名列表组成。其实更通用的描述是, 一条声明语句由一个基本数据类型 (base type) 和紧随其后的一个声明符 (declarator) 列表组成。每个声明符命名了一个变量并指定该变量为与基本数据类型有关的某种类型。

目前为止, 我们所接触的声明语句中, 声明符其实就是变量名, 此时变量的类型也就是声明的基本数据类型。其实还可能有更复杂的声明符, 它基于基本数据类型得到更复杂的类型, 并把它指定给变量。

2.3.1 引用



C++11 中新增了一种引用: 所谓的“右值引用 (rvalue reference)”, 我们将在 13.6.1 节 (第 471 页) 做更详细的介绍。这种引用主要用于内置类。严格来说, 当我们使用术语“引用 (reference)”时, 指的其实是“左值引用 (lvalue reference)”。

引用 (reference) 为对象起了另外一个名字, 引用类型引用 (refers to) 另外一种类型。通过将声明符写成 `&d` 的形式来定义引用类型, 其中 `d` 是声明的变量名:

```
int ival = 1024;
int &refVal = ival;    // refVal 指向 ival (是 ival 的另一个名字)
int &refVal2;          // 报错: 引用必须被初始化
```



50



一般在初始化变量时，初始值会被拷贝到新建的对象中。然而定义引用时，程序把引用和它的初始值绑定（bind）在一起，而不是将初始值拷贝给引用。一旦初始化完成，引用将和它的初始值对象一直绑定在一起。因为无法令引用重新绑定到另外一个对象，因此引用必须初始化。



引用即别名



引用并非对象，相反的，它只是为一个已经存在的对象所起的另外一个名字。

定义了一个引用之后，对其进行的所有操作都是在与之绑定的对象上进行的：

```
refVal = 2;           // 把 2 赋给 refVal 指向的对象，此处即是赋给了 ival
int ii = refVal;      // 与 ii = ival 执行结果一样
```

51

为引用赋值，实际上是把值赋给了与引用绑定的对象。获取引用的值，实际上是获取了与引用绑定的对象的值。同理，以引用作为初始值，实际上是以与引用绑定的对象作为初始值：

```
// 正确：refVal3 绑定到了那个与 refVal 绑定的对象上，这里就是绑定到 ival 上
int &refVal3 = refVal;
// 利用与 refVal 绑定的对象的值初始化变量 i
int i = refVal; // 正确：i 被初始化为 ival 的值
```

因为引用本身不是一个对象，所以不能定义引用的引用。

引用的定义

允许在一条语句中定义多个引用，其中每个引用标识符都必须以符号&开头：

```
int i = 1024, i2 = 2048; // i 和 i2 都是 int
int &r = i, r2 = i2;      // r 是一个引用，与 i 绑定在一起，r2 是 int
int i3 = 1024, &ri = i3;  // i3 是 int，ri 是一个引用，与 i3 绑定在一起
int &r3 = i3, &r4 = i2;    // r3 和 r4 都是引用
```

除了 2.4.1 节（第 55 页）和 15.2.3 节（第 534 页）将要介绍的两种例外情况，其他所有引用的类型都要和与之绑定的对象严格匹配。而且，引用只能绑定在对象上，而不能与字面值或某个表达式的计算结果绑定在一起，相关原因将在 2.4.1 节详述：

```
int &refVal4 = 10;        // 错误：引用类型的初始值必须是一个对象
double dval = 3.14;
int &refVal5 = dval;      // 错误：此处引用类型的初始值必须是 int 型对象
```

2.3.1 节练习

练习 2.15：下面的哪个定义是不合法的？为什么？

- (a) `int ival = 1.01;` (b) `int &rval1 = 1.01;`
 (c) `int &rval2 = ival;` (d) `int &rval3;`

练习 2.16：考查下面的所有赋值然后回答：哪些赋值是不合法的？为什么？哪些赋值是合法的？它们执行了什么样的操作？

- ```
int i = 0, &r1 = i; double d = 0, &r2 = d;
(a) r2 = 3.14159; (b) r2 = r1;

(c) i = r2; (d) r1 = d;
```

练习 2.17: 执行下面的代码段将输出什么结果?

```
int i, &ri = i;
i = 5; ri = 10;
std::cout << i << " " << ri << std::endl;
```

### 2.3.2 指针



52

指针 (pointer) 是“指向 (point to)”另外一种类型的复合类型。与引用类似, 指针也实现了对其他对象的间接访问。然而指针与引用相比又有很多不同点。其一, 指针本身就是一个对象, 允许对指针赋值和拷贝, 而且在指针的生命周期内它可以先后指向几个不同的对象。其二, 指针无须在定义时赋初值。和其他内置类型一样, 在块作用域内定义的指针如果没有被初始化, 也将拥有一个不确定的值。



指针通常难以理解, 即使是有经验的程序员也常常因为调试指针引发的错误而被备受折磨。

定义指针类型的方法将声明符写成\*d的形式, 其中d是变量名。如果在一条语句中定义了几个指针变量, 每个变量前面都必须有符号\*:

```
int *ip1, *ip2; // ip1 和 ip2 都是指向 int 型对象的指针
double dp, *dp2; // dp2 是指向 double 型对象的指针, dp 是 double 型对象
```

#### 获取对象的地址

指针存放某个对象的地址, 要想获取该地址, 需要使用取地址符 (操作符&):

```
int ival = 42;
int *p = &ival; // p 存放变量 ival 的地址, 或者说 p 是指向变量 ival 的指针
```

第二条语句把p定义为一个指向int的指针, 随后初始化p令其指向名为ival的int对象。因为引用不是对象, 没有实际地址, 所以不能定义指向引用的指针。

除了2.4.2节 (第56页) 和15.2.3节 (第534页) 将要介绍的两种例外情况, 其他所有指针的类型都要和它所指向的对象严格匹配:

```
double dval;
double *pd = &dval; // 正确: 初始值是 double 型对象的地址
double *pd2 = pd; // 正确: 初始值是指向 double 对象的指针

int *pi = pd; // 错误: 指针 pi 的类型和 pd 的类型不匹配
pi = &dval; // 错误: 试图把 double 型对象的地址赋给 int 型指针
```

因为在声明语句中指针的类型实际上被用于指定它所指向对象的类型, 所以二者必须匹配。如果指针指向了一个其他类型的对象, 对该对象的操作将发生错误。

#### 指针值

指针的值 (即地址) 应属下列4种状态之一:

1. 指向一个对象。
2. 指向紧邻对象所占空间的下一个位置。
3. 空指针, 意味着指针没有指向任何对象。
4. 无效指针, 也就是上述情况之外的其他值。

试图拷贝或以其他方式访问无效指针的值都将引发错误。编译器并不负责检查此类错误，这一点和试图使用未经初始化的变量是一样的。访问无效指针的后果无法预计，因此程序员必须清楚任意给定的指针是否有效。

尽管第2种和第3种形式的指针是有效的，但其使用同样受到限制。显然这些指针没有指向任何具体对象，所以试图访问此类指针（假定的）对象的行为不被允许。如果这样做了，后果也无法预计。

### 利用指针访问对象

如果指针指向了一个对象，则允许使用解引用符（操作符\*）来访问该对象：

```
int ival = 42;
int *p = &ival; // p 存放着变量 ival 的地址，或者说 p 是指向变量 ival 的指针
cout << *p; // 由符号*得到指针 p 所指的变量，输出 42
```

对指针解引用会得出所指的对象，因此如果给解引用的结果赋值，实际上也就是给指针所指的对象赋值：

```
*p = 0; // 由符号*得到指针 p 所指的变量，即可经由 p 为变量 ival 赋值
cout << *p; // 输出 0
```

如上述程序所示，为\*p赋值实际上是为p所指的对象赋值。

Note

解引用操作仅适用于那些确实指向了某个对象的有效指针。

### 关键概念：某些符号有多重含义

像&和\*这样的符号，既能用作表达式里的运算符，也能作为声明的一部分出现，符号的上下文决定了符号的意义：

```
int i = 42;
int &r = i; // &紧随类型名出现，因此是声明的一部分，r 是一个引用
int *p; // *紧随类型名出现，因此是声明的一部分，p 是一个指针
p = &i; // &出现在表达式中，是一个取地址符
*p = i; // *出现在表达式中，是一个解引用符
int &r2 = *p; // &是声明的一部分，*是一个解引用符
```

在声明语句中，&和\*用于组成复合类型；在表达式中，它们的角色又转变成运算符。在不同场景下出现的虽然是同一个符号，但是由于含义截然不同，所以我们完全可以把它当作不同的符号来看待。

### 空指针

空指针（null pointer）不指向任何对象，在试图使用一个指针之前代码可以首先检查它是否为空。以下列出几个生成空指针的方法：

54

```
int *p1 = nullptr; // 等价于 int *p1 = 0;
int *p2 = 0; // 直接将 p2 初始化为字面常量 0
// 需要首先#include cstdlib
int *p3 = NULL; // 等价于 int *p3 = 0;
```

C++  
11

得到空指针最直接的办法就是用字面值 **nullptr** 来初始化指针，这也是 C++11 新标准刚刚引入的一种方法。nullptr 是一种特殊类型的字面值，它可以被转换成（参见 2.1.2 节，

第 32 页) 任意其他的指针类型。另一种办法就如对 p2 的定义一样, 也可以通过将指针初始化为字面值 0 来生成空指针。

过去的程序还会用到一个名为 NULL 的预处理变量 (preprocessor variable) 来给指针赋值, 这个变量在头文件 cstdlib 中定义, 它的值就是 0。

2.6.3 节 (第 68 页) 将稍微介绍一点关于预处理器的知识, 现在只要知道预处理器是运行于编译过程之前的一段程序就可以了。预处理变量不属于命名空间 std, 它由预处理器负责管理, 因此我们可以直接使用预处理变量而无须在前面加上 std::。

当用到一个预处理变量时, 预处理器会自动地把它替换为实际值, 因此用 NULL 初始化指针和用 0 初始化指针是一样的。在新标准下, 现在的 C++ 程序最好使用 nullptr, 同时尽量避免使用 NULL。

把 int 变量直接赋给指针是错误的操作, 即使 int 变量的值恰好等于 0 也不行。

```
int zero = 0;
pi = zero; // 错误: 不能把 int 变量直接赋给指针
```

### 建议: 初始化所有指针

使用未经初始化的指针是引发运行时错误的一大原因。

和其他变量一样, 访问未经初始化的指针所引发的后果也是无法预计的。通常这一行为将造成程序崩溃, 而且一旦崩溃, 要想定位到出错位置将是特别棘手的问题。

在大多数编译器环境下, 如果使用了未经初始化的指针, 则该指针所占内存空间的当前内容将被看作一个地址值。访问该指针, 相当于去访问一个本不存在的位置上的本不存在的对象。糟糕的是, 如果指针所占内存空间中恰好有内容, 而这些内容又被当作了某个地址, 我们就很难分清它到底是合法的还是非法的了。

因此建议初始化所有的指针, 并且在可能的情况下, 尽量等定义了对象之后再定义指向它的指针。如果实在不清楚指针应该指向何处, 就把它初始化为 nullptr 或者 0, 这样程序就能检测并知道它没有指向任何具体的对象了。

### 赋值和指针

指针和引用都能提供对其他对象的间接访问, 然而在具体实现细节上二者有很大不同, 其中最重要的一点就是引用本身并非一个对象。一旦定义了引用, 就无法令其再绑定到另外的对象, 之后每次使用这个引用都是访问它最初绑定的那个对象。

◀ 55

指针和它存放的地址之间就没有这种限制了。和其他任何变量 (只要不是引用) 一样, 给指针赋值就是令它存放一个新的地址, 从而指向一个新的对象:

```
int i = 42;
int *pi = 0; // pi 被初始化, 但没有指向任何对象
int *pi2 = &i; // pi2 被初始化, 存有 i 的地址
int *pi3; // 如果 pi3 定义于块内, 则 pi3 的值是无法确定的

pi3 = pi2; // pi3 和 pi2 指向同一个对象 i
pi2 = 0; // 现在 pi2 不指向任何对象了
```

有时候要想搞清楚一条赋值语句到底是改变了指针的值还是改变了指针所指对象的值不太容易, 最好的办法就是记住赋值永远改变的是等号左侧的对象。当写出如下语句时,

```
pi = &ival; // pi 的值被改变, 现在 pi 指向了 ival
```



意思是给 `pi` 赋一个新的值，也就是改变了那个存放在 `pi` 内的地址值。相反的，如果写出如下语句，

```
*pi = 0; // ival 的值被改变，指针 pi 并没有改变
```

则 `*pi`（也就是指针 `pi` 指向的那个对象）发生改变。

### 其他指针操作

只要指针拥有一个合法值，就能将它用在条件表达式中。和采用算术值作为条件（参见 2.1.2 节，第 32 页）遵循的规则类似，如果指针的值是 0，条件取 `false`：

```
int ival = 1024;
int *pi = 0; // pi 合法，是一个空指针
int *pi2 = &ival; // pi2 是一个合法的指针，存放着 ival 的地址
if (pi) // pi 的值是 0，因此条件的值是 false
 // ...
if (pi2) // pi2 指向 ival，因此它的值不是 0，条件的值是 true
 // ...
```

任何非 0 指针对应的条件值都是 `true`。

对于两个类型相同的合法指针，可以用相等操作符（`==`）或不相等操作符（`!=`）来比较它们，比较的结果是布尔类型。如果两个指针存放的地址值相同，则它们相等；反之它们不相等。这里两个指针存放的地址值相同（两个指针相等）有三种可能：它们都为空、都指向同一个对象，或者都指向了同一个对象的下一地址。需要注意的是，一个指针指向某对象，同时另一个指针指向另外对象的下一地址，此时也有可能出现这两个指针值相同的情况，即指针相等。

因为上述操作要用到指针的值，所以不论是作为条件出现还是参与比较运算，都必须使用合法指针，使用非法指针作为条件或进行比较都会引发不可预计的后果。

3.5.3 节（第 105 页）将介绍更多关于指针的操作。

### 56 void\* 指针

**void\*** 是一种特殊的指针类型，可用于存放任意对象的地址。一个 `void*` 指针存放着一个地址，这一点和其他指针类似。不同的是，我们对该地址中到底是个什么类型的对象并不了解：

```
double obj = 3.14, *pd = &obj; // 正确：void* 能存放任意类型对象的地址
void *pv = &obj; // obj 可以是任意类型的对象
pv = pd; // pv 可以存放任意类型的指针
```

利用 `void*` 指针能做的事儿比较有限：拿它和别的指针比较、作为函数的输入或输出，或者赋给另外一个 `void*` 指针。不能直接操作 `void*` 指针所指的對象，因为我们并不知道这个对象到底是什么类型，也就无法确定能在这个对象上做些哪些操作。

概括说来，以 `void*` 的视角来看内存空间也就仅仅是内存空间，没办法访问内存空间中所存的对象，关于这点将在 19.1.1 节（第 726 页）有更详细的介绍，4.11.3 节（第 144 页）将讲述获取 `void*` 指针所存地址的方法。

## 2.3.2 节练习

练习 2.18: 编写代码分别更改指针的值以及指针所指对象的值。

练习 2.19: 说明指针和引用的主要区别。

练习 2.20: 请叙述下面这段代码的作用。

```
int i = 42;
int *p1 = &i;
*p1 = *p1 * *p1;
```

练习 2.21: 请解释下述定义。在这些定义中有非法的吗？如果有，为什么？

```
int i = 0;
(a) double* dp = &i; (b) int *ip = i; (c) int *p = &i;
```

练习 2.22: 假设 p 是一个 int 型指针，请说明下述代码的含义。

```
if (p) // ...
if (*p) // ...
```

练习 2.23: 给定指针 p，你能知道它是否指向了一个合法的对象吗？如果能，叙述判断的思路；如果不能，也请说明原因。

练习 2.24: 在下面这段代码中为什么 p 合法而 lp 非法？

```
int i = 42; void *p = &i; long *lp = &i;
```

## 2.3.3 理解复合类型的声明



如前所述，变量的定义包括一个基本数据类型（base type）和一组声明符。在同一条定义语句中，虽然基本数据类型只有一个，但是声明符的形式却可以不同。也就是说，一条定义语句可能定义出不同类型的变量：

```
// i 是一个 int 型的数，p 是一个 int 型指针，r 是一个 int 型引用
int i = 1024, *p = &i, &r = i;
```



很多程序员容易迷惑于基本数据类型和类型修饰符的关系，其实后者不过是声明符的一部分罢了。

## 定义多个变量



经常有一种观点会误以为，在定义语句中，类型修饰符（\*或&）作用于本次定义的全部变量。造成这种错误看法的原因有很多，其中之一是我们可以把空格写在类型修饰符和变量名中间：

```
int* p; // 合法但是容易产生误导
```

我们说这种写法可能产生误导是因为 int\* 放在一起好像是这条语句中所有变量共同的类型一样。其实恰恰相反，基本数据类型是 int 而非 int\*。\* 仅仅是修饰了 p 而已，对该声明语句中的其他变量，它并不产生任何作用：

```
int* p1, p2; // p1 是指向 int 的指针，p2 是 int
```

涉及指针或引用的声明，一般有两种写法。第一种把修饰符和变量标识符写在一起：

```
int *p1, *p2; // p1 和 p2 都是指向 int 的指针
```

这种形式着重强调变量具有的复合类型。第二种把修饰符和类型名写在一起，并且每条语句只定义一个变量：

```
int* p1; // p1 是指向 int 的指针
int* p2; // p2 是指向 int 的指针
```

这种形式着重强调本次声明定义了一种复合类型。



上述两种定义指针或引用的不同方法没有孰对孰错之分，关键是选择并坚持其中的一种写法，不要总是变来变去。

本书采用第一种写法，将\*（或是&）与变量名连在一起。

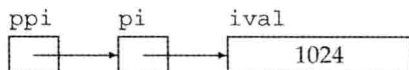
### 指向指针的指针

一般来说，声明符中修饰符的个数并没有限制。当有多个修饰符连写在一起时，按照其逻辑关系详加解释即可。以指针为例，指针是内存中的对象，像其他对象一样也有自己的地址，因此允许把指针的地址再存放到另一个指针当中。

通过\*的个数可以区分指针的级别。也就是说，\*\*表示指向指针的指针，\*\*\*表示指向指针的指针的指针，以此类推：

```
int ival = 1024;
int *pi = &ival; // pi 指向一个 int 型的数
int **ppi = π // ppi 指向一个 int 型的指针
```

此处 pi 是指向 int 型数的指针，而 ppi 是指向 int 型指针的指针，下图描述了它们之间的关系。



解引用 int 型指针会得到一个 int 型的数，同样，解引用指向指针的指针会得到一个指针。此时为了访问最原始的那个对象，需要对指针的指针做两次解引用：

```
cout << "The value of ival\n"
 << "direct value: " << ival << "\n"
 << "indirect value: " << *pi << "\n"
 << "doubly indirect value: " << **ppi
 << endl;
```

该程序使用三种不同的方式输出了变量 ival 的值：第一种直接输出；第二种通过 int 型指针 pi 输出；第三种两次解引用 ppi，取得 ival 的值。

### 指向指针的引用

引用本身不是一个对象，因此不能定义指向引用的指针。但指针是对象，所以存在对指针的引用：

```
int i = 42;
int *p; // p 是一个 int 型指针
int *&r = p; // r 是一个对指针 p 的引用

r = &i; // r 引用了一个指针，因此给 r 赋值&i 就是令 p 指向 i
*r = 0; // 解引用 r 得到 i，也就是 p 指向的对象，将 i 的值改为 0
```



要理解 `r` 的类型到底是什么，最简单的办法是从右向左阅读 `r` 的定义。离变量名最近的符号（此例中是 `&r` 的符号 `&`）对变量的类型有最直接的影响，因此 `r` 是一个引用。声明符的其余部分用以确定 `r` 引用的类型是什么，此例中的符号 `*` 说明 `r` 引用的是一个指针。最后，声明的基本数据类型部分指出 `r` 引用的是一个 `int` 指针。



面对一条比较复杂的指针或引用的声明语句时，从右向左阅读有助于弄清楚它的真实含义。

### 2.3.3 节练习

59

练习 2.25: 说明下列变量的类型和值。

(a) `int* ip, i, &r = i;` (b) `int i, *ip = 0;` (c) `int* ip, ip2;`

## 2.4 const 限定符



有时我们希望定义这样一种变量，它的值不能被改变。例如，用一个变量来表示缓冲区的大小。使用变量的好处是当我们觉得缓冲区大小不再合适时，很容易对其进行调整。另一方面，也应随时警惕防止程序一不小心改变了这个值。为了满足这一要求，可以用关键字 **const** 对变量的类型加以限定：

```
const int bufSize = 512; // 输入缓冲区大小
```

这样就把 `bufSize` 定义成了一个常量。任何试图为 `bufSize` 赋值的行为都将引发错误：

```
bufSize = 512; // 错误：试图向 const 对象写值
```

因为 `const` 对象一旦创建后其值就不能再改变，所以 `const` 对象必须初始化。一如既往，初始值可以是任意复杂的表达式：

```
const int i = get_size(); // 正确：运行时初始化
const int j = 42; // 正确：编译时初始化
const int k; // 错误：k 是一个未经初始化的常量
```

### 初始化和 const

正如之前反复提到的，对象的类型决定了其上的操作。与非 `const` 类型所能参与的操作相比，`const` 类型的对象能完成其中大部分，但也不是所有的操作都适合。主要的限制就是只能在 `const` 类型的对象上执行不改变其内容的操作。例如，`const int` 和普通的 `int` 一样都能参与算术运算，也都能转换成一个布尔值，等等。

在不改变 `const` 对象的操作中还有一种是初始化，如果利用一个对象去初始化另外一个对象，则它们是不是 `const` 都无关紧要：

```
int i = 42;
const int ci = i; // 正确：i 的值被拷贝给了 ci
int j = ci; // 正确：ci 的值被拷贝给了 j
```

尽管 `ci` 是整型常量，但无论如何 `ci` 中的值还是一个整型数。`ci` 的常量特征仅仅在执行改变 `ci` 的操作时才会发挥作用。当用 `ci` 去初始化 `j` 时，根本无须在意 `ci` 是不是一个常量。拷贝一个对象的值并不会改变它，一旦拷贝完成，新的对象就和原来的对象没什么关系了。